

Software-as-a-Graph: Benchmarking Dependency Counts and Graph Learning for Pre-Deployment Cascade-Impact Ranking in Publish–Subscribe Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe middleware hides the paths along which failures cascade, so architects lack pre-deployment evidence on which components are systemically critical. Software-as-a-Graph (SaG) models event-driven architectures as typed multigraphs, derives explicit dependency graphs from deployment manifests, and ranks components by simulated cascade impact. We benchmark closed-form centrality, dependency counts, graph neural networks and hybrid engines on twelve synthetic architectures under leave-one-scenario-out evaluation and on five hand-authored models of open-source systems, against three simulators. The registered primary contrast, a heterogeneous graph transformer against weighted centrality, was null (Spearman $\Delta\rho = +0.069$, $p = 0.27$). In exploratory analyses, the number of a component’s direct dependents, which is publish–subscribe afferent coupling and a typed two-hop query on the raw graph, ranked reachability-simulated impact at $\rho = 0.764$ and beat centrality on all twelve architectures; transitive dependents reached 0.938 on the system models. Because that simulator’s first propagation wave is exactly this count, we also scored a queue-flow simulator, on which the count kept a partial correlation of 0.259 after the reachability ranking was removed, and a multi-criteria simulator, on which weighted centrality ranked above it (0.702 vs. 0.650). Catching 80% of the true top-fifth required flagging the top 45%. Graph attention networks on the dependency graph matched the count (0.748) but never exceeded it. Counting took 1–29 ms per architecture, against 0.08–4.5 s for one simulation. The result is a benchmark with a largely negative outcome for graph learning: once dependencies are explicit, a classic coupling metric is the baseline to beat.

Keywords: Dependency graphs, cascading failures, publish–subscribe, graph neural networks, software architecture, dependability, empirical study

1. Introduction

1.1. Motivation and Problem

Large distributed systems increasingly communicate through asynchronous publish–subscribe (pub-sub) middleware: ROS 2 in autonomous driving [1], Apache Kafka in enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub decouples producers and consumers in space, time and synchronization [7]. Components interact through topics and brokers rather than direct references, and deployment-time Quality-of-Service (QoS) policies govern reliability, durability, priority and deadlines.

The same decoupling hides how failures spread. Publishers and subscribers share no direct link, so outages, head-of-line blocking and backpressure propagate along concealed paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. These failures take two forms. In *sequential cascades*,

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

a slow subscriber fills a broker queue and gradually starves its publishers [10]. In *simultaneous blasts*, a shared-library crash or host outage takes down every colocated service at once. Neither architecture diagrams nor static call graphs show these mechanisms. The cheapest time to reduce the risk is before deployment, at design and continuous-integration time [11, 12], when no runtime telemetry exists. Architects therefore need to know, from configuration manifests alone, which components, topics and links are systemically critical and why.

Existing practice leaves this gap open, which we call the **Architecture–Code Gap**: a system can have bug-free code in every service and still be fragile through hidden single points of failure or mismatched QoS contracts [13, 14]. Architecture evaluations such as ATAM rely on manual elicitation [15]. Static code analysis inspects services in isolation [16, 17]. Chaos engineering [18] needs a provisioned cluster, while pre-production alternatives like service-level fault injection testing [19] and microservice dependency tracing [20] require runnable execution environments. Homogeneous centrality flattens typed topologies into untyped graphs [21, 22]. Learned models, graph neural networks in particular, could combine these structural cues. What is missing is a representation that makes pub-sub failure paths explicit, and evidence on which analyzer to trust on it: whether graph learning adds anything over the coupling metrics that architecture research already has. This paper provides that evidence for one well-defined task, ranking components by simulated cascade impact before deployment.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment static analysis framework for event-driven architectures (Figure 1). It (1) models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1); (2) derives from it, through publish–subscribe rules, an explicit `DEPENDS_ON` dependency graph (Section 3.3); and (3) ranks components by predicted cascade impact with training-free rankers, graph-learning engines, and hybrid engines in which a learned model corrects a closed-form score (Section 4). Predictors read only the analysis graph. Ground truth comes from three simulators that run on the raw structural topology (Section 4.3).

What the ground truth is, and is not. All ground truth in this study is simulated. The primary oracle I^* is a reachability simulation that runs on the same manifest the predictors read, and its first propagation wave is exactly the dependency count this paper recommends (Section 4.4). The study therefore measures agreement between cheap static rankers and simulators, not the prediction of observed outages. We bound the circularity with two further simulators and with partial correlations, and report the cost of running I^* itself (Section 7.4); validation against observed failures remains open (Section 8.3).

Findings in brief. The registered primary contrast, a heterogeneous graph transformer against QoS-weighted centrality, was null ($+0.069$, $p = 0.266$). In exploratory analyses registered after it, the simplest ranker won: the number of a component’s direct dependents (**InDeg**), which is publish–subscribe afferent coupling [23, 24], ranks I^* at $\rho = 0.764$ and beats centrality on all twelve synthetic architectures. It is a typed two-hop query on the raw graph (Proposition 1): untyped raw-graph metrics do not recover it, but merely counting the topics a component publishes comes within 0.033 of it. Graph attention networks on the dependency graph match the count (0.748) but never exceed it. On a queue-flow simulator the count keeps a partial correlation of 0.259 after the reachability oracle’s rank is removed (on a 30-Application sample per fold); on a multi-criteria simulator QoS-weighted centrality ranks above it. Flagging enough components to catch 80% of the true top-20% set takes the top 45% by the count. The result is a benchmark with a largely negative outcome for graph learning on this task.

1.3. Research Questions

- **RQ1 (Ranking accuracy):** *How accurately do SaG’s closed-form, learned and hybrid engines rank components by cascading-failure impact on unseen architectures, compared with structural baselines and with dependency counts on the derived graph?*
- **RQ2 (What learning needs):** *What do learned engines need: which graph they read, relation-specific (typed) parameters, or QoS inputs, once model capacity and edge-channel width are matched?*
- **RQ3 (Transfer):** *How well do engines trained on synthetic architectures transfer zero-shot to independently authored models inspired by five open-source systems?*

- **RQ4 (Cost):** *What does the analysis cost at CI/CD time, which stage dominates, and how does it compare with running the simulation directly?*

The evaluation follows a registered protocol. Only the primary contrast, the matched 2×2 and the two hybrid contrasts are confirmatory; the dependency counts, the dependency-graph learners, the raw-graph baselines and the cross-oracle analyses were registered in later amendments, after the primary result was known, and are reported as exploratory (Section 6.3). Supplementary Section S25 logs every amendment and maps these four research questions onto the registered plan.

1.4. Contributions

Evaluated under leave-one-scenario-out (LOSO) cross-validation over twelve synthetic architectures and on five hand-authored models of open-source systems, this paper contributes:

1. **A benchmark of static rankers for pre-deployment cascade-impact ranking in publish-subscribe systems** (Sections 6–7): seventeen architectures (2,812 components), three simulation oracles, and nineteen training-free, learned and hybrid rankers plus a first-order reference, with a registered analysis plan, its amendments, and artifacts that regenerate byte-identically.
2. **A largely negative result for graph learning on this task** (Sections 7.1–7.2). Afferent coupling, computed as a typed two-hop count, ranks the reachability oracle at $\rho = 0.764$ and beats closed-form centrality on 12/12 architectures (+0.211); graph neural networks gain +0.08 to +0.11 by reading the dependency graph instead of the raw multigraph, reaching 0.748, but never exceed the count, and on the raw multigraph they act as per-node models. Relation typing adds nothing at matched capacity (main effect -0.014), and in the single untuned configuration tested the heterogeneous transformer is unstable across seeds.
3. **A bounded account of what the count measures** (Sections 4.4 and 7.1). Its agreement with the reachability oracle is largely construction, and the paper says so; on a queue-flow simulator it retains $\rho = 0.610$, and a partial correlation of 0.259 [0.143, 0.367] beyond the reachability oracle, on a 30-Application sample per fold; on a multi-criteria simulator it is outranked by QoS-weighted centrality (0.650 vs. 0.702). Its critical-set recall is moderate: the top 45% is needed to catch 80% of the true top 20%.
4. **Explicit dependency rules for publish-subscribe architectures** (Section 3), which state the typed query behind afferent coupling and add library-mediated blasts; the library rule raises transitive reach by +0.058.
5. **A cost profile** (Section 7.4): on the corpus the counting path (projection plus count) takes 1–29 ms per architecture against 0.08–4.5 s for one run of the reachability oracle, and neural feature extraction costs median $5.6 \times$ the oracle.

A previous conference paper [25] introduced the preliminary multigraph and deterministic quality model on synthetic topologies. This paper adds the dependency rules’ evaluation, the dependency counts, the learned and hybrid engines, the QoS edge encoding, LOSO and zero-shot evaluation, the matched control, the cross-oracle analyses and the cost profile; the earlier quality model survives only as the proposed explanation layer of Section 5.

Section 2 reviews related work, Sections 3–5 present the model, the rankers and the proposed explanation layer, Sections 6–7 the evaluation, Section 8 the discussion and threats, and Section 9 concludes.

2. Related Work

2.1. Dependability Analysis of Distributed Systems

Runtime approaches to dependability, such as broker clustering, backpressure, autoscaling, failover and chaos engineering [18], require a running cluster, can disrupt service, and consume substantial compute. That compute is itself a concern of green software engineering [26, 27, 28]. Pre-production alternatives, including service-level fault injection testing [19] and microservice dependency localization via PageRank (MicroRank

[20]), provide lighter options before deployment. Architecture-based reliability prediction has a long history. Yacoub and Ammar’s methodology for architecture-level reliability risk analysis [29] pioneered combining component dependency graphs with complexity and failure severity to rank components by risk. Cheung’s absorbing Markov chain [30] and the state-, path- and additive models surveyed by Goseva-Popstojanova and Trivedi [31] and Immonen and Niemelä [32] are classic examples, as are the Palladio Component Model [33], layered queueing networks [34] and the AADL Error Model Annex [35]. These approaches answer broader questions but need operational profiles and failure rates that are unavailable at commit time. SaG asks a narrower question from manifests alone: whose failure reaches furthest in the declared topology?

Telemetry-driven methods diagnose faults in running microservices. Examples are Seer [36] and Sage [37], MicroRCA [38], TraceRCA [39], and GNN-based DeepTraLog [40], Eadro [41] and MicroCause [42] (reviewed in [43]). In synchronous microservices, cascades stem from thread-pool exhaustion, RPC timeouts and retry storms [44]. In pub-sub systems they spread through queue saturation and message starvation. All of these methods need traces or metrics from a live system; SaG works before any code runs. SaG’s question is also that of change impact analysis [45] and of dependency management in microservice fleets [46]. Ranking by impact relates to learning-to-rank defect prediction [47], which optimizes the ranking measure directly, as our listwise loss does (Section 4.2).

2.2. Static Code Analysis and Static System Analysis

Static code analysis (SCA) tools such as SonarQube [16] measure complexity [48], cohesion and coupling [17, 49] within individual services to flag defect-prone modules [50, 51, 52, 53]. Comparing sophisticated graph measures against simple structural metrics has a rich history in software engineering defect prediction: Zimmermann and Nagappan [54] evaluated network analysis on dependency graphs, while Premraj and Herzig’s replication study [55] demonstrated that simple code and coupling metrics largely matched network measures. SCA cannot see inter-service messaging, broker saturation or cross-host propagation. Architecture recovery tools reconstruct system-level structure from code [56, 57], and HAROS [58] checks ROS systems statically before launch. SaG’s static system analysis (SSA) uses the declared topology instead, propagating code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, as afferent coupling [23], the Absolute Importance of the Service [24] and service fan-in [59]. In publish–subscribe systems these counts are not a single edge lookup, because consumers reach producers only through topics, brokers and shared libraries. They are a typed two-hop query (publisher $\rightarrow$ topic $\leftarrow$ subscriber), which SaG’s dependency rules state explicitly; Section 7.1 shows that this count, not the untyped degree or PageRank of the same raw graph, is what ranks simulated cascade impact. This lets teams find structural anti-patterns [60, 61] and architectural technical debt [62] in CI/CD [63, 64].

2.3. Quality Models and Multi-Criteria Evaluation

ISO/IEC 25010:2023 [65] and ISO/IEC 25019:2023 [66] define product quality and quality in use. SaG covers the characteristics derivable from deployment topology, namely Availability, Fault Tolerance and the Maintainability sub-characteristics (Section 5.1), and links internal structural quality to external dependability [67, 68]. Aggregating metrics into an auditable score is a multi-criteria decision problem, for which the Analytic Hierarchy Process (AHP) [69] is standard. AHP’s consistency ratio detects inconsistent judgments but not matrices back-filled from a chosen answer, a distinction this study reports for its weights (Supplementary Section S4).

2.4. Graph Learning and Explainability

Centrality indices [21, 22, 70, 71] and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. A single untyped score conflates structurally different elements, such as topics, libraries and hosts, and cannot say *why* a component is critical. Learned node-importance methods, including FINDER [72], DrBC [73] and PowerGraph [74], share the homogeneity assumption. Homogeneous GNNs (GCN [75], GraphSAGE [76], GAT [77]) discard relation identity unless it is supplied as a feature. Heterogeneous GNNs (RGCN [78], HAN [79], HGT [80], MAGNN [81]) learn relation-specific transformations. We evaluate both families under matched capacity (Section 7.2). Khodabandeh et al. [82] apply graph

attention to microservice call graphs to predict future interactions. We instead predict the impact of removing a node from a declared topology. GNN explainers such as GNNExplainer [83] and PGExplainer [84] explain models in terms of their internal features. SaG’s explanation layer (Section 5) instead names ISO/IEC quality sub-characteristics and a remediation class for each flagged component.

This study belongs with work showing that simple baselines are often competitive with sophisticated learners [85, 86, 87], and, in software engineering specifically, with the finding that network measures on dependency graphs [54] add little over simple coupling metrics once those are measured [55]. Its contribution is of that kind: a benchmark, with a largely negative result for graph learning, on a pre-deployment ranking task where the simple baseline is a known architecture metric.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 presents the end-to-end architecture of the SaG framework. The shared front end processes Architecture-as-Code manifests, constructs a typed multigraph, projects implicit runtime interactions throughout a logical dependency layer, and extracts typed node properties. These features feed the ranking engines (Section 4) and, separately and with no shared parameters, the explanation layer (Section 5).

3.1. Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.
- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality and connection strength. For Applications and Libraries, $w_V(v) = 1 - \text{CQP}(v)$, where CQP is a code-quality penalty computed from static code metrics (lines of code, cyclomatic complexity, coupling and cohesion); otherwise $w_V(v) = 1.0$.

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

3.2. Quality-of-Service Link Weighting

A link’s strength depends on its Quality-of-Service (QoS) contract: a **RELIABLE** topic with **TRANSIENT_LOCAL** durability couples services more strongly than a **BEST_EFFORT** telemetry stream. Each topic t carries an aggregate weight $w(t) \in (0, 1]$ combining declared QoS policies (reliability, durability, priority) with payload size and publication frequency. The sub-weights of reliability, durability and priority come from an Analytic Hierarchy Process (AHP) pairwise-comparison matrix that was stated independently rather than back-solved from a target vector ($CR = 0.016$, non-degenerate; Supplementary Section S4, which also shows that three of the framework’s other AHP matrices do encode a declared vector).

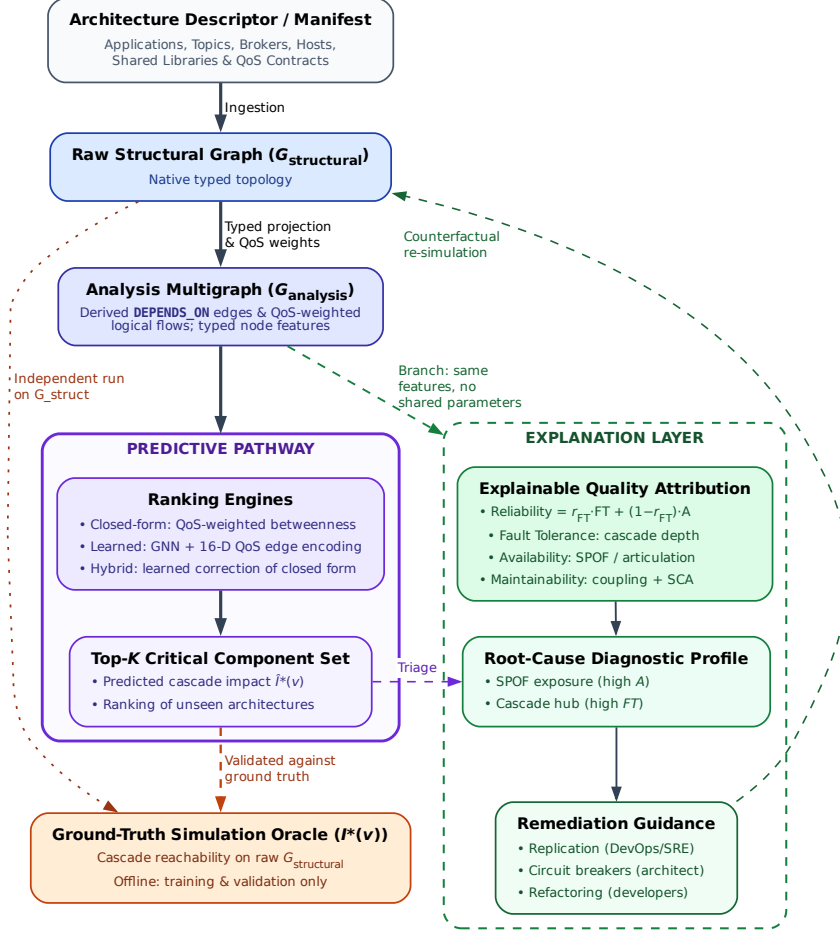


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway runs down the centre: manifest ingestion, typed multigraph, `DEPENDS_ON` projection with typed node properties, the ranking engines (closed-form, learned and hybrid; Figure 3), and the ranked critical set. The dashed edge marks the ground-truth simulation oracles, which operate only on $G_{\text{structural}}$, train the predictor offline and take no part in inference. The proposed explanation layer (Section 5, not evaluated) reads the same analysis multigraph, shares no parameters with the predictor, and is applied to components after they have been ranked; no output of the predictor flows into it.

Table 2: Notation used throughout the paper.

Symbol	Description
$G_{\text{structural}}$	Raw multigraph (used exclusively by simulation oracles)
G_{analysis}	Logical <code>DEPENDS_ON</code> projection (input to predictors)
V_{app}	Application nodes (the primary scored population)
$w(t), w(e)$	QoS topic weight and derived dependency edge weight
$I^*(v)$	Primary cascade-reachability simulation oracle
$I_{\text{dyn}}(v)$	Dynamic queue-flow discrete-event simulation oracle
$I_{\text{comp}}(v)$	Multi-criteria composite simulation oracle
$Q(v)$	Reliability–Maintainability composite quality score
ρ	Spearman rank correlation coefficient (full population)
$\rho_{>0}$	Spearman rank correlation restricted to active stratum ($I > 0$)
$\text{Overlap}@K$	Top- K identification set overlap ($K = 0.20 V_{\text{app}} $)

On the reachability oracle, QoS weighting does not improve closed-form ranking: unweighted betweenness on the Application–Library projection scores $\rho = 0.591$, against 0.553 for QoS-weighted betweenness (Topo-QoS; Section 7.1). On the multi-criteria oracle the order reverses, and Topo-QoS is the best training-free ranker (Section 7.1). The dependency counts evaluated in this paper are unweighted and do not read $w(t)$.

3.3. Logical Dependency Projection (*DEPENDS_ON*)

Structural edges do not directly reflect failure propagation: a subscriber depends on a publisher, yet no direct edge joins them in pub-sub topologies. SaG derives an explicit semantic relation, *DEPENDS_ON*, directed from *dependent* to *dependency* (“if the target fails, the source is impacted”), via the rules of Table 3.

Table 3: The *DEPENDS_ON* logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive USES)	$1 - \prod_{t \in T} (1 - w(t))$
2	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_{\text{DEPENDS_ON}}(u \rightarrow v)$
4	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_{\text{DEPENDS_ON}}(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6	broker_to_broker	Broker $\leftrightarrow$ Broker (shared fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine topics T joining a pair by probabilistic union [88, 89, 90], ensuring parallel failure paths increase coupling. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [91], and Rules 3 and 4 lift dependencies to hosts by maximum.

Sequential cascades and simultaneous blasts. Rule 1 captures sequential cascades, where a failed publisher starves subscribers through queues and buffers. Rule 5 captures simultaneous blasts, where a crashed library takes down all dependent applications at once. Libraries that publish or subscribe are endpoints of Rule 1 in their own right, and an Application reaches a library’s topics transitively through Rule 5. Rules 2, 3, 4 and 6 represent infrastructural and broker dependencies; they are defined for completeness but are not exercised by the Application-level evaluation in this paper.

Proposition 1 (dependency count = typed two-hop count). Let G_{flow} be the Application–Library projection (Rules 1 and 5) and v an Application. Then

$$\text{InDeg}(v) = |\{u \neq v : \exists t \in V_{\text{topic}}, (v, t) \in \text{PUBLISHES_TO} \wedge (u, t) \in \text{SUBSCRIBES_TO}\}|. \quad (2)$$

Proof. Rule 5 edges end at Libraries, so every edge into an Application is a Rule 1 edge $u \rightarrow v$, which exists exactly when $u \neq v$ subscribes to a topic that v publishes; parallel topics collapse into one edge. $\square$

The right-hand side of Eq. (2) is publish–subscribe afferent coupling (fan-in, the Absolute Importance of the Service [23, 24]), and it is a typed two-hop query on the raw multigraph. *InDeg* therefore needs no projection to compute; the projection states which typed query to ask. The equality is also checked on all seventeen corpus graphs (maximum absolute difference 0). What the projection adds beyond this query is measured separately: Rule 5 raises transitive reach by +0.058 (Section 7.1).

3.4. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the derived *DEPENDS_ON* edges and code metrics (Figure 2). Predictor features are computed on G_{analysis} , while simulation oracles run strictly on $G_{\text{structural}}$ (Section 4.4).

3.5. Typed Node Feature Encoding

Both the predictive pathway (Section 4) and the explanation layer (Section 5) read typed node properties from G_{analysis} . All five entity types share an 18-dimensional base block of normalized topological metrics ($[0, 1]$): PageRank, Reverse PageRank, betweenness, closeness, eigenvector centrality, in- and out-degree, clustering, articulation, bridge ratio, the node QoS weight, incoming and outgoing dependency weights, multi-path coupling, path complexity, fan-out criticality, and the Connectivity Degradation Index (CDI). Four centrality features (PageRank, Reverse PageRank, betweenness and closeness) are computed over weighted

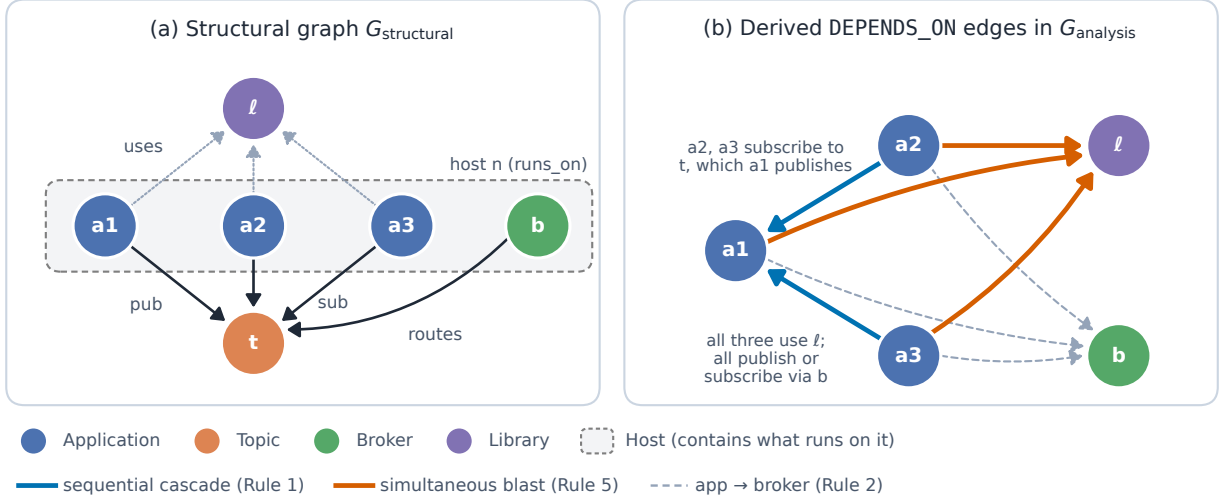


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library l , and all run on host n . No structural edge joins two applications. (b) The derived DEPENDS_ON edges make the hidden dependencies explicit: subscribers $a2, a3$ depend on publisher $a1$ (Rule 1), all applications depend on l (Rule 5), and each application depends on the broker (Rule 2). Simulation oracles run on view (a); predictors read view (b).

edges and so carry QoS information; the nominal QoS-off condition is therefore not completely QoS-free. The in-degree feature means that every learner is given a quantity closely related to InDeg as an input (Section 7.2). Type-specific blocks extend the vector to 19–25 dimensions (full schema in Supplementary Section S11). CDI evaluates structural connectivity loss via a fixed-size breadth-first sample to control computation time (Section 7.4); its sensitivity to hash-seeded tie breaking is examined in Section 8.3.

4. Ranking Engines and Ground Truth

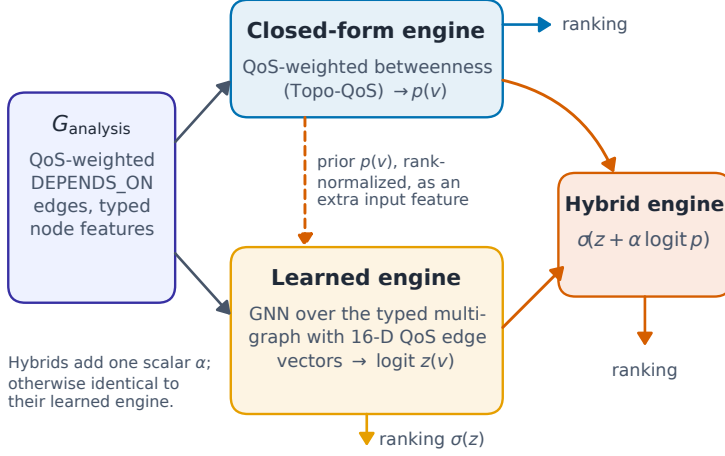
SaG ranks components with three kinds of engine. The **closed-form engine** Topo-QoS is QoS-weighted betweenness on the dependency projection (Section 6.2). The **learned engines** are graph neural networks over the typed multigraph and the derived dependency graph (this section). The **hybrid engines** are learned engines that correct the closed-form score (Section 7.1). All are trained or scored against simulation oracles that run on a separate graph view (Sections 4.3–4.4). Figure 3 shows how the three engines relate and how they are evaluated. Full hyperparameters and training commands are on the experiment pages of the replication repository (Section 6.1).

4.1. Heterogeneous Graph Transformer and Attention Networks

The primary learned engine HGT-QoS is a three-layer Heterogeneous Graph Transformer (HGT) [80] in PyTorch Geometric [92], with hidden dimension $D = 64$ and $H = 4$ heads. Entity-specific linear projections map raw node features $x_v \in \mathbb{R}^{19-25}$ into hidden space, $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys, queries, and values, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ (full layer-wise formulations in Supplementary Section S1.1). Message passing runs over both G_{analysis} and its transpose. On the raw multigraph, native relations point away from Applications, so forward message passing cannot reach scored nodes, reducing forward GNNs to per-node MLPs over precomputed centralities (Section 8.2). The homogeneous Graph Attention Network baseline (GAT-QoS , and GAT-P-QoS on the dependency projection) uses a 3-layer architecture with 4 attention heads and width $D = 288$ ($D = 296$ in the unweighted control to match parameter capacity), projecting all node types into a shared embedding space before homogeneous GATConv layers.

Each directed edge carries a 16-dimensional vector $e_{uv} \in \mathbb{R}^{16}$: index 0 represents the coupling weight $w_E(e)$, index 1 the normalized path count, indices 2–8 a relation one-hot, and indices 9–15 middleware QoS

(a) Three ranking engines, one analysis graph



(b) Ground truth and evaluation

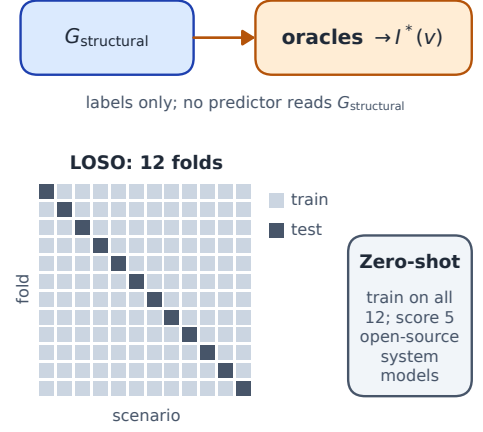


Figure 3: (a) SaG’s three ranking engines read the analysis graph. The closed-form engine scores QoS-weighted betweenness $p(v)$; the learned engine outputs a logit $z(v)$. A hybrid engine gives the learned engine $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \logit p)$, with one learnable scalar α . (b) Ground truth comes from simulation oracles on the structural graph, which no predictor reads. Engines are evaluated by leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five open-source system models.

parameters (reliability, durability, priority, depart-mode flag, deadline pair, max-blocking time). The edge vector is projected and added before attention, $\tilde{h}_v = h_v + W_{\text{edge}} e_{uv}$.

4.2. Prediction Head and Training Objective

A composite head predicts simulated cascade impact, $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{a}_1(v) \parallel \hat{a}_2(v)))$, where auxiliary heads $\hat{a}_1, \hat{a}_2$ provide feature enrichment ($\hat{a}_1$ supervised on I^* , $\hat{a}_2$ unsupervised). The objective combines regression with listwise and pairwise ranking:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (3)$$

where $\mathcal{L}_{\text{rank}}$ is ListMLE [93] over the ground-truth permutation π ,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (4)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss ($\gamma = 0.05$) over pairs differing by more than γ . The learned engines and the explanation layer share no parameters (Supplementary Section S20).

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) under cosine warm restarts for up to 300 epochs, with early stopping at patience 30 on an inner validation split, over five fixed seeds {42, 123, 456, 789, 2024}. Hyperparameters and loss coefficients were fixed before the first sweep and are identical across arms; no arm was tuned and no hyperparameter search was run, so every learned result below is conditional on this one configuration (Section 8.3).

4.3. Ground-Truth Simulation Oracles

Ground truth is evaluated using simulation oracles on the raw structural multigraph $G_{\text{structural}}$:

- **Primary reachability cascade oracle (I^*):** Crashes component v , propagates outages through dependent topics, brokers and links via breadth-first traversal on $G_{\text{structural}}$, and computes the mean fractional feed loss across intact subscribers. Feed loss is scaled by a QoS severity ladder ($\times 1.2$ RELIABLE,

×1.15 high priority, ×1.05 medium) and clamped to [0, 1]. Mean across five tie-breaking seeds forms the label. Disabling QoS scaling leaves the Application ordering largely intact ($\rho = 0.965$ across the twelve folds), and substituting durability-aware rescaling moves it less still ($\rho = 0.977$; Supplementary Section S9), showing that I^* is predominantly a topological reachability metric.

- **Independent queue-flow discrete-event oracle (I_{dyn}):** A discrete-event SimPy [94] simulation modeling dynamic message emission rates, queue buffer saturation, and network latency. It records the drop in delivered message rate suffered by surviving consumers under active load, and it reads the declared rates, payload sizes and QoS contracts that I^* ignores. Its mechanism is therefore different from reachability, but its substrate is the same pub-sub topology, and its first-order effect—consumers that lose a failed publisher’s messages—is again subscriber loss. Discrete-event simulation of every Application is expensive (about 15 s per fault run on the larger folds), and I_{dyn} is available only for the first 30 Applications of each fold in lexicographic order (all 26 on ATM), seed 42. That sample is arbitrary and may correlate with generation order; a full-population, five-seed labelling was registered (Amendment 11) but not completed, so every I_{dyn} result in this paper rests on 26–30 Applications per fold. On this sample I_{dyn} agrees with I^* at mean $\rho = 0.627$, and its published test–retest reliability is 0.74–0.97 (Supplementary Section S9), which bounds the correlation any deterministic ranker can attain with it.
- **Composite multi-criteria failure oracle (I_{comp}):** The Validate-stage failure simulator, run exhaustively over all 1,321 Applications with QoS weighting on and a primed baseline flow. It scores each removal as $I_{\text{comp}}(v) = 0.35 \text{RL}(v) + 0.25 \text{FR}(v) + 0.25 \text{TL}(v) + 0.15 \text{FD}(v)$: reachability loss, fragmentation (0.70 structural component loss + 0.30 severity), throughput loss and flow disruption, with AHP-derived weights. Because fragmentation and flow disruption respond to almost every removal, $I_{\text{comp}}(v) > 0$ for virtually every Application. I_{comp} is never a training label.

4.4. Input–Label Independence and Construction Bounds

Features are extracted strictly from G_{analysis} , while simulation oracles execute on $G_{\text{structural}}$, and no simulation output is exposed as an input feature; a regression test in the replication package enforces this.

Procedural separation does not remove construct overlap. $I^*(v)$ propagates failure along the same subscriber→publisher and application→library relations that the dependency rules state, and in its first propagation wave the set of affected subscribers is exactly what **InDeg** counts (Proposition 1). Agreement between **InDeg** and I^* therefore largely measures how closely the count reproduces the simulator’s own propagation rule, and the first-order expansion of I^* (Eq. (6)) is reported beside it as a reference. Two analyses bound this circularity rather than remove it. First, every ranker is also scored against I_{dyn} and I_{comp} . Second, because I_{dyn} shares the topology and the first-order effect, its agreement with a ranker is also reported after the rank of I^* (and, separately, of the first-order term) has been partialled out (Section 7.1). Neither analysis replaces validation against observed failures, which this study does not have (Section 8.3).

5. A Proposed Explanation Layer (Not Evaluated)

A ranking says where risk is highest, not how to reduce it. A component may be critical because it is an unreplicated single point of failure, an error-propagating cascade hub, or a high-coupling maintainability bottleneck, and each calls for a different intervention: replication, circuit breakers, or decoupling. SaG includes a layer that attributes a flagged component to one of these causes. **This layer is a design proposal. Its attributions have not been validated against practitioner judgment or against the outcome of applying the remediation it names, and nothing in Sections 6–8 evaluates it.** It is described here because it is part of the released tool and because its inputs overlap the rankers’ features; it is not one of this paper’s contributions, and it is not used as a ranker (as a ranker it is weak: its composite reaches $\rho = 0.200$ against I^* at the shipped setting; Supplementary Section S1).

5.1. Structure

Following ISO/IEC 25010:2023 [65] and ISO/IEC 25019:2023 [66], the layer profiles each component along **Fault Tolerance** (FT), which informs circuit breakers and redundancy; **Availability** (A), which informs replication; and **Maintainability** (M), which informs decoupling and refactoring. Each is a weighted sum of rank-normalized graph metrics read from the same node properties as the rankers (Section 3.5): Reverse PageRank, in-degree and cascade depth for FT ; articulation severity, QoS-weighted single-point-of-failure severity, bridge ratio and the Connectivity Degradation Index for A ; betweenness, efferent coupling, code-quality penalty and clustering for M . The composite $Q(v) = 0.80 R(v) + 0.20 M(v)$, with $R = 0.36 FT + 0.64 A$, flags components above the Tukey upper fence of Q (mean 4.2% of components), and the $FT/A/M$ profile names the remediation class (Figure 4; example card: Supplementary Section S19). The full formulas, the AHP weights and their consistency diagnostics are in Supplementary Sections S24 and S4. Three of those weight vectors encode a declared priority vector rather than independent elicitation, and moving the intra-dimension weights toward the elicited judgment lowers the layer’s rank correlation ($0.319 \rightarrow 0.200$; Supplementary Section S1). Validating the attributions is the layer’s principal open question (Section 8.4).

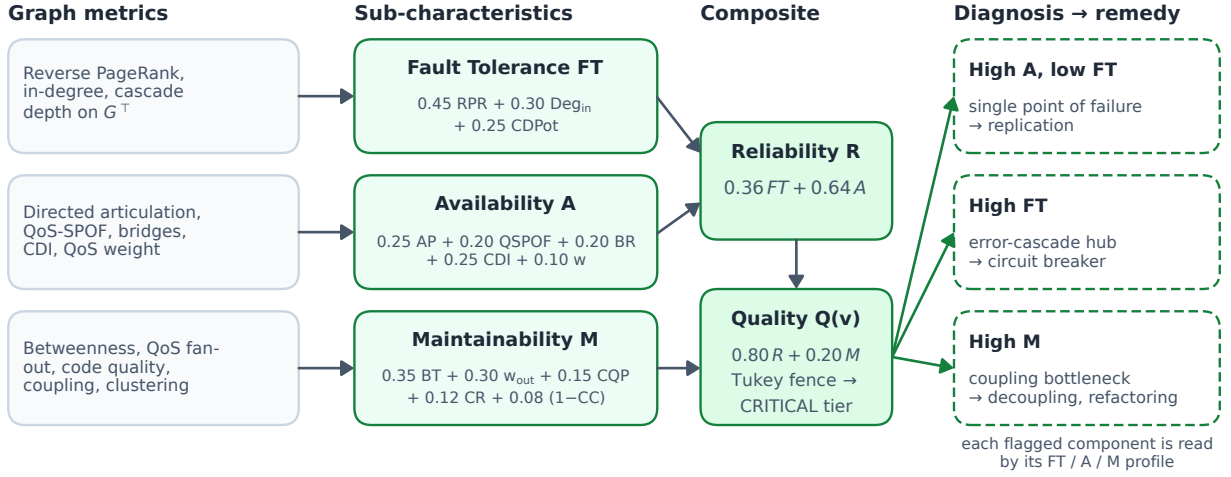


Figure 4: The proposed explanation layer (not evaluated in this paper). Rank-normalized graph metrics feed the ISO/IEC 25010 sub-characteristics Fault Tolerance, Availability and Maintainability (CR: coupling risk; CC: clustering coefficient), which combine into Reliability and the composite $Q(v)$. A component above the Tukey fence of Q is flagged, and its $FT/A/M$ profile names the remediation class.

6. Experimental Setup

6.1. Corpus and Replication Package

The corpus comprises 2,812 components across seventeen architectures (Table 4). Twelve synthetic topologies form the LOSO folds. They span autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration and air-traffic management. All twelve come from one generator family, as do their code metrics, so LOSO measures transfer across configurations of that generator. Each regenerates byte-identically from a committed configuration, and CI verifies this against a SHA-256 manifest.

The five open-source systems are hand-authored models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes inspired by Online Boutique and Train-Ticket were each authored by one author as typed multigraphs from public documentation; they are not mechanical extractions. Brokers, QoS profiles, code metrics and host specifications are partly assumed. Two models depart materially from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, whereas the original is an RPC-based service mesh with no broker, and the Train-Ticket model represents its service-discovery

Table 4: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; the five open-source system models are excluded from all training and used only for zero-shot transfer (Section 7.3). Counts are read from the committed topology files and verified in CI; per-scenario composition: Supplementary Section S13.

Dataset	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

server as a broker. Neither contains synchronous call edges. RQ3 therefore tests transfer to independently authored architecture models inspired by open-source systems, not to running deployed systems.

Replication package. Datasets, harnesses, checkpoints and result artifacts are archived on Zenodo (see Data Availability). The public repository documents each experiment: its protocol, hyperparameters, `make` target, artifacts and the supplementary section holding its extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>).

6.2. Predictors

Table 5 lists the predictors: training-free rankers on the raw multigraph, which need no derivation; training-free rankers on the derived graphs (**Topo**, **Topo-QoS**, **InDeg**, **Reach**); learned engines on the raw multigraph and on the dependency graph; and hybrid engines. On a GNN, **-QoS** means the 16-D QoS edge vector (Section 4.1) together with three QoS node columns (w , w_{in} , w_{out}), and **Hybrid-X** is engine X corrected by the closed-form prior. **GAT** and **GAT-QoS** are untyped GATs matched to HGT in parameter budget, and **GAT-QoS** reads the same 16-D edge vector as **HGT-QoS**, so the four learned models form a 2×2 over typing (GAT vs. HGT) and the QoS channel. Smaller and projection-based variants that the study also ran are listed in Supplementary Section S30. The learned and hybrid engines ingest the native typed multigraph under LOSO, and the **-P** learners the Application-Library **DEPENDS_ON** graph, with bit-identical node features and labels. In QoS-off arms every edge weight is 1 and the QoS node columns are zeroed, though four centralities still carry QoS (Section 3.5, Supplementary Section S11).

Table 5: Predictors reported in this paper. **-QoS**: QoS-weighted distances (Topo) or the 16-D QoS edge vector plus three QoS node columns (GNNs); **-P**: trained on the dependency graph; **Hybrid-X**: engine X corrected by the **Topo-QoS** prior; **GAT-P+InDeg**: **GAT-P-QoS** corrected by the **InDeg** prior. Each dependency-graph learner keeps its raw-multigraph counterpart’s architecture and parameter budget. All runs, with substrate details: Supplementary Section S30.

Predictor	Graph read	Typing / edge input	Params
<i>Training-free, raw multigraph (no derivation; Amendment 12)</i>			
Degree-raw, PR-raw, RevPR-raw	$G_{\text{structural}}$	untyped	0
Pubs-raw (topics published)	$G_{\text{structural}}$	typed 1-hop	0
Reach-R1 (pub→topic→sub closure)	$G_{\text{structural}}$	typed 2-hop	0
<i>Training-free, derived graphs</i>			
Topo	application-layer graph	untyped	0
Topo-QoS	App-Lib DEPENDS_ON	scalar $w(e)$	0
InDeg (Eq. 2)	App-Lib DEPENDS_ON	none	0
Reach	App-Lib DEPENDS_ON	none	0
<i>Learned, raw multigraph: typing $\times$ QoS channel at matched budget</i>			
GAT / GAT-QoS	$G_{\text{structural}}$ + features	homogeneous; none / 16-D	437,496 / 429,992
HGT / HGT-QoS	$G_{\text{structural}}$ + features	heterogeneous; 1-hot / 16-D	434,620
Hybrid-GAT / Hybrid-HGT	$G_{\text{structural}}$ + features	as base + Topo-QoS prior	431,433 / 434,941
<i>Learned, dependency graph (Amendment 9)</i>			
GAT-P, GAT-P-QoS, HGT-P-QoS	App-Lib DEPENDS_ON	as counterpart	as counterpart
GAT-P+InDeg	App-Lib DEPENDS_ON	as GAT-P-QoS + InDeg prior	as counterpart

Closed-form scores. Topo-QoS is computed on the Application-Library `DEPENDS_ON` graph (Rules 1 and 5), because on the raw multigraph messages route through topics and brokers and Application betweenness vanishes; the registered comparator Topo reads betweenness from the analysis stage’s application-layer graph:

$$\text{Topo}(v) = 0.6 \cdot \text{BT}(v) + 0.4 \cdot \text{AP}(v), \quad \text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (5)$$

where BT is normalized betweenness, BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so strongly coupled edges attract shortest paths, and AP flags articulation points. **Defect in the registered comparator.** Because of an implementation defect, the articulation term reads zero for every node, so the reported Topo and Topo-QoS rank exactly as betweenness and QoS-weighted betweenness. They are kept as registered for the confirmatory tests. Restoring the term lowers both (Topo $0.349 \rightarrow 0.329$, Topo-QoS $0.553 \rightarrow 0.533$; Supplementary Section S22), so the defect does not favour the counts. The two therefore differ in substrate as well as in weighting, and the registered controls of Section 7.1 separate the two.

Dependency counts. $\text{InDeg}(v)$ is the number of direct dependents of v on the Application-Library `DEPENDS_ON` graph, which for an Application equals publish-subscribe afferent coupling (Proposition 1), and $\text{Reach}(v)$ is the number of its transitive dependents, normalized by $|V| - 1$.

Raw-multigraph baselines. To separate the value of choosing the right metric from the value of the derivation (Amendment 12), five rankers are computed on $G_{\text{structural}}$ without any derived edge: total degree (Degree-raw), the number of topics a component publishes (Pubs-raw), unweighted PageRank ($\alpha = 0.85$) on the raw graph and on its reverse (PR-raw, RevPR-raw), and Reach-R1, the transitive closure of publisher $\rightarrow$ topic $\rightarrow$ subscriber paths (Reach without Rule 5).

6.3. Metrics, Protocols and Statistics

Population. Every predictor is scored on the Application set V_{app} .

Metrics. Ranking is measured by Spearman ρ against the simulation oracles. To separate rank ordering among active components from the trivial agreement on inert components with zero impact, we report both full-population Spearman ρ and active-stratum Spearman $\rho_{>0}$ (restricting to components with true impact > 0). Critical-set identification is measured by $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K with $K = \text{round}(0.20 |V_{\text{app}}|)$. PR-AUC, $F_1@T$ and nDCG@10 are reported in Supplementary Section S15.

Protocols. Under *LOSO*, learned models train on eleven scenarios and are tested on the twelfth, over all 12 folds and five seeds; the reported value is the mean over seeds of each seed’s ρ . Under *zero-shot transfer*, models trained on all twelve scenarios are evaluated without fine-tuning on the five system models. Training-free rankers are fitted to nothing, so for them both protocols reduce to *per-scenario evaluation* on the same twelve scenarios and five models; they are placed in the same tables only so that every row is scored on identical scenarios and labels.

Statistics. Summary ρ is the arithmetic mean of the per-fold (or per-system) Spearman correlations, with a 95% CI from a bootstrap that resamples folds ($B = 2,000$) [95]; comparisons use paired two-sided Wilcoxon signed-rank tests over folds [96]. With 12 folds, the smallest attainable two-sided p is 0.00049. Fisher- z averaging leaves the order of every row of Table 6 unchanged. Weighting folds by $|V_{\text{app}}|$ leaves the top five unchanged (InDeg rises to 0.810) but lifts Topo-QoS to 0.596, level with the raw-multigraph learned engines (0.595–0.606) (Supplementary Section S35). Spearman ρ assigns tied scores their average rank. $\text{Overlap}@K$ breaks ties at the K boundary by node order, which is arbitrary for integer counts; Section 7.1 therefore also reports tie-aware recall curves.

Registered analysis plan: what is confirmatory and what is not. Before the twelve-fold harness produced any result, we registered the primary contrast, HGT-QoS vs. Topo-QoS, together with HGT vs. Topo-QoS and Holm correction across the two. We call it *registered* because the plan is committed, with dates, in our repository. Three amendments added confirmatory contrasts: the matched 2×2 (Amendment 2), Hybrid-HGT (Amendment 5) and Hybrid-GAT (Amendment 6). **The results this paper leads with are not confirmatory.** The dependency counts and QoS controls (Amendment 7, 2026-09-25), the dependency-graph learners (Amendment 9) and the value of the derivation (Amendment 10, both 2026-09-26) were each registered before their own arms were run, but after the primary sweep had returned its null result,

and with the choice of comparators informed by it; a full-population labelling of I_{dyn} (Amendment 11) was registered but not completed, and the raw-graph baselines, partial correlations, learned-engine rescoring and recall analyses (Amendment 12) were registered after this paper’s previous review round. We report them as exploratory, with Holm correction within each family and none in the omnibus. Pooling all thirteen registered confirmatory contrasts under an omnibus Holm correction (`reproduce/omnibus_holm.py`), both hybrid primaries remain significant (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$), while the primary contrast HGT-QoS vs Topo-QoS does not reach significance ($+0.069$, nominal $p = 0.266$, $p_{\text{omni}} \geq 0.46$). Supplementary Section S25 lists all amendments with dates and outcomes.

7. Results

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, and, for the rankers of Table 7, also against the queue-flow oracle $I_{\text{dyn}}(v)$ and the multi-criteria oracle $I_{\text{comp}}(v)$ (Section 4.3). Everything except the primary contrast, the matched 2×2 and the two hybrid contrasts is exploratory (Section 6.3). Per-fold results, secondary strata and extended protocol notes are in the Supplementary Material and public experiment pages (Section 6.1). Figure 5 summarizes the main findings.

7.1. RQ1: Ranking Accuracy Against Structural Baselines

Summary. Counting a component’s direct dependents (**InDeg**), which is publish–subscribe afferent coupling, ranks I^* at $\rho = 0.764$ and beats closed-form centrality (**Topo-QoS**, 0.553) on all twelve synthetic architectures ($+0.211$, Holm $p = 0.002$, exploratory). It is computable on the raw multigraph as a typed two-hop query; untyped raw-graph metrics fall far below it, but simply counting the topics a component publishes comes within 0.033. A dependency-graph GAT matches it (0.748) without exceeding it. On I_{dyn} the count keeps a partial correlation of 0.259 after the rank of I^* is removed; on I_{comp} it loses to QoS-weighted centrality. Recovering 80% of the true top-20% set requires flagging the top 45% by **InDeg**.

Learned engines are trained on eleven scenarios and scored on the twelfth; training-free rankers are simply scored on each of the twelve (26 to 300 Applications, K between 5 and 60). Table 6 reports all of them against I^* .

The count, and what the derivation adds to it. **InDeg** reaches $\rho = 0.764$ ($\rho_{>0} = 0.516$) and beats **Topo-QoS** on every fold; **Reach**, the number of transitive dependents, reaches 0.732 (11/12). By Proposition 1, **InDeg** is publish–subscribe afferent coupling [23, 24, 59] and can be computed on the raw multigraph as a typed two-hop query. The raw-graph rankers of Table 6 separate what the choice of that query contributes from what the derived graph contributes. Untyped raw-graph metrics fail: total degree reaches 0.199 and reverse PageRank 0.089, and PageRank is constant for every Application because Applications have no incoming raw edges (**InDeg** ahead on 12/12 folds, Holm $p = 0.006$ each). A trivial typed count does not fail: the number of topics a component publishes (**Pubs-raw**) reaches 0.731, only 0.033 below **InDeg** (11/12 folds, Holm $p = 0.002$; Amendment 10, corrected in this revision, Supplementary Section S25). **The $+0.211$ margin over centrality is therefore a property of choosing afferent coupling as the metric, not of the dependency projection.** What the projection adds beyond the typed query is measured in one place: without Rule 5, **Reach** falls by 0.058 (9/12 folds, Holm $p = 0.0068$; Amendment 10).

Learners on the dependency graph match the count. The same graph neural networks, trained on the dependency graph instead of the raw multigraph, reach $\rho = 0.748$ (**GAT-P-QoS**), level with **InDeg** (-0.017 , Holm $p = 1.000$). Adding the **InDeg** score as a prior yields 0.758 (**GAT-P+InDeg**): learning a correction to the count adds nothing measurable to it. Every learner already receives in-degree as a node feature (Section 3.5); a learner without that feature was not run, so whether message passing alone would recover the count is untested. HGT-P-QoS reaches only 0.514 and is highly seed-dependent (mean within-fold seed SD 0.254; Section 8.3).

Hybrids outperform closed-form centrality, not the count. Hybrid-HGT reaches $\rho = 0.657$ ($+0.103$, Holm $p = 0.0068$) and Hybrid-GAT reaches $\rho = 0.683$ ($+0.130$, Holm $p = 0.0029$), each on 11 of 12 folds. Both survive omnibus Holm correction across all thirteen confirmatory contrasts ($p_{\text{omni}} = 0.041$ and 0.019). These are the study’s only significant confirmatory results. On their own, the raw-multigraph learned engines

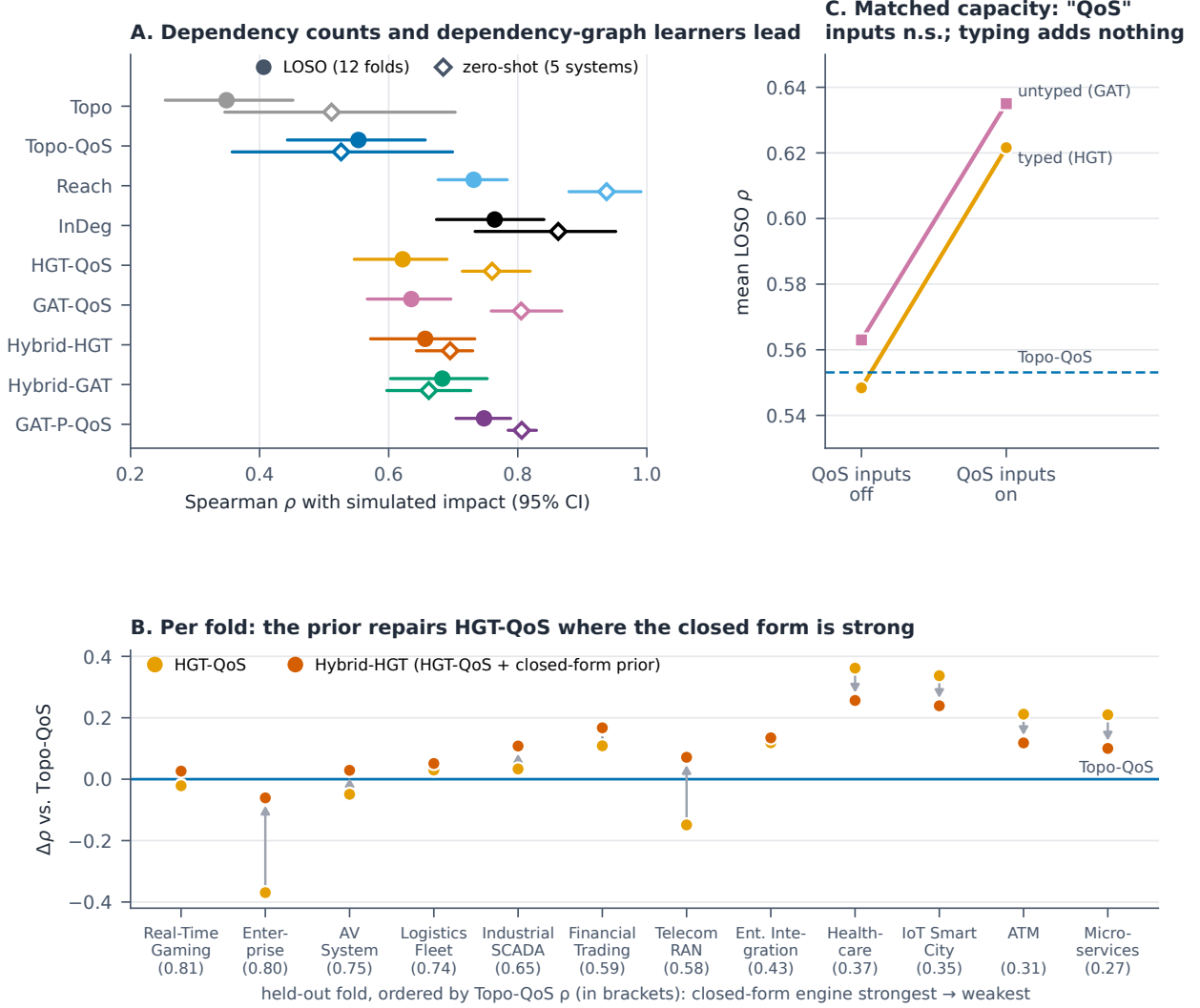


Figure 5: Main results at a glance, Application population. (A) Mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; Table 6) and zero-shot on five system models (open diamonds). Counting dependents on the derived dependency graph (**InDeg**, **Reach**) achieves top performance on both unseen synthetic architectures and system models; neural models reading the dependency graph reach parity with **InDeg**. (B) Per held-out fold, the gain of HGT-QoS and of Hybrid-HGT over **Topo-QoS**. (C) Cell means of the capacity- and channel-matched 2×2 (Table 8): the “QoS” inputs, which include a QoS-weighted in-degree column, raise both models by about 0.07 (not significant after Holm), while typed and untyped configurations are indistinguishable.

Table 6: Main results under LOSO across twelve synthetic architectures (Application population; learned engines: five seeds, CPU sweeps). $\Delta\rho$ is paired by fold against **Topo-QoS**, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon signed-rank test. $\rho_{>0}$ denotes Spearman correlation restricted to active components ($I^* > 0$). p_{Holm} is given within each registered family: hybrids (Amendments 5 and 6, confirmatory) and dependency counts (Amendment 7, exploratory). [‡]Exploratory: Amendment 9 registered its contrasts against **InDeg**, **Reach** and raw counterparts. [§]Raw-multigraph rankers (Amendment 12, exploratory; unadjusted p); PR-raw is constant for every Application and has no defined ρ . [¶]**Topo** and **Topo-QoS** carry an implementation defect (articulation term zero; Section 6.2); corrected values are 0.329 and 0.533. The reference row is the post hoc first-order expansion of I^* (Eq. (6)); *outside every registered family). Per-fold values: Supplementary Sections S23, S33, S34 and S35.

Predictor	LOSO ρ [95% CI]	Active $\rho_{>0}$	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K
<i>Reference: first-order expansion of I^* (post hoc)</i>						
Analytic I^{**}	0.808 [0.755, 0.853]	0.631	+0.255 [+0.170, +0.344]	12/12	0.0005*	0.536
<i>Training-free, raw multigraph (no derivation)[§]</i>						
Degree-raw	0.199 [0.068, 0.328]	0.218	−0.354 [−0.475, −0.236]	1/12	0.0015	0.307
RevPR-raw	0.089 [−0.019, 0.192]	0.075	−0.465 [−0.558, −0.364]	0/12	0.0005	0.199
Pubs-raw	0.731 [0.637, 0.810]	0.410	+0.178 [+0.098, +0.266]	11/12	0.0015	0.487
Reach-R1	0.674 [0.606, 0.738]	0.088	+0.121 [+0.012, +0.220]	9/12	0.064	0.300
<i>Training-free, application layer</i>						
Topo[¶]	0.349 [0.254, 0.452]	0.174	−0.204 [−0.286, −0.122]	0/12	0.0005	0.366
<i>Training-free, dependency graph</i>						
Topo-QoS[¶]	0.553 [0.443, 0.657]	0.280	—	—	—	0.388
Reach	0.732 [0.674, 0.782]	0.286	+0.178 [+0.088, +0.268]	11/12	0.0034 (0.0068)	0.344
InDeg (pub-sub fan-in)	0.764 [0.674, 0.840]	0.516	+0.211 [+0.132, +0.299]	12/12	0.0005 (0.0020)	0.504
<i>Learned and hybrid, on the raw multigraph</i>						
HGT-QoS	0.622 [0.547, 0.690]	0.312	+0.069 [−0.046, +0.174]	8/12	0.266	0.426
GAT-QoS	0.635 [0.567, 0.696]	0.338	+0.082 [−0.046, +0.201]	7/12	0.233	0.438
Hybrid-HGT	0.657 [0.572, 0.733]	0.345	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435
Hybrid-GAT	0.683 [0.603, 0.753]	0.362	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450
<i>Learned, on the dependency graph (Amendment 9)</i>						
GAT-P-QoS	0.748 [0.704, 0.789]	0.440	+0.195 [+0.100, +0.294]	10/12	0.0068 [‡]	0.454
GAT-P+InDeg	0.758 [0.710, 0.801]	0.537	+0.205 [+0.112, +0.298]	11/12	0.0034 [‡]	0.468
HGT-P-QoS	0.514 [0.392, 0.636]	0.237	−0.039 [−0.155, +0.077]	4/12	0.622 [‡]	0.380

do not differ significantly from **Topo-QoS** (HGT-QoS +0.069, $p = 0.266$, the null primary contrast; GAT-QoS +0.082, $p = 0.233$), and both hybrids rank below **InDeg** (0.657 and 0.683 against 0.764).

Where the closed-form gain comes from. **Topo** reads betweenness from the application-layer graph, whereas **Topo-QoS** computes QoS-weighted betweenness on the Application–Library dependency graph. Three controls registered in Amendment 7 separate substrate from weighting: unweighted betweenness on the same graph scores 0.591; constant topic weights score 0.595; permuting QoS profiles scores 0.559 (−0.006, $p = 0.73$). On I^* , the gain comes from the Application–Library graph structure, not from QoS contract content.

The first-order expansion of the oracle. To show how much of **InDeg**’s agreement with I^* comes from the simulator’s own propagation rule, Table 6 includes a post hoc closed-form first-order expansion of I^* :

$$\hat{I}_1^*(v) = \sum_{t \in \text{pub}(v)} \frac{|\text{sub}(t)|}{|\text{pub}(t)|}, \quad (6)$$

where $|\text{pub}(t)| \geq 1$ holds for all published topics $t \in \text{pub}(v)$ by construction (v publishes to t), preventing any division by zero. It reaches mean $\rho = 0.808$ [0.755, 0.853] ($\rho_{>0} = 0.631$): direct subscriber loss is most of what I^* measures, and **InDeg** is a coarser version of the same quantity. As a reference it bounds the mean rather than every fold: on four folds (AV, Financial Trading, Healthcare, Industrial SCADA) **InDeg** slightly exceeds it (by 0.005–0.038).

Beyond the reachability oracle. Table 7 scores the rankers against the two other oracles. I_{dyn} agrees with I^* only moderately ($\rho = 0.627$ [0.495, 0.749] on the same sample), and its published test–retest reliability is 0.74–0.97 (Supplementary Section S9), which bounds what any deterministic ranker can reach. On I_{dyn} , **InDeg** reaches $\rho = 0.610$ and beats **Topo-QoS** on 11 of 12 folds (+0.217, $p = 0.0034$). Because I_{dyn} shares the topology and the first-order effect with I^* , raw agreement cannot show that the count captures anything the reachability oracle does not; the partial correlation can. After the rank of I^* is removed, **InDeg** keeps

Table 7: Rankers against three oracles, twelve LOSO folds, Application population (exploratory). I^* and I_{comp} are exhaustive over all 1,321 Applications. I_{dyn} covers only the first 30 Applications of each fold in lexicographic order (all 26 on ATM), seed 42; the full-population labels registered in Amendment 11 were not produced (Section 8.3). Partial ρ : Spearman correlation with I_{dyn} after the rank of I^* is regressed out of both (Amendment 12). Learned rows (Amendment 12, R3) score the mean of the five seeds’ predictions, which raises ρ on I^* above the per-seed means of Table 6; their I^* column is on that basis. 95% CIs bootstrap folds.

Ranker	I^* ρ	I_{dyn} ρ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} I^*)$ [95% CI]	I_{comp} ρ
<i>Training-free</i>				
Analytic I^*	0.808	0.636 [0.526, 0.731]	0.276 [0.180, 0.375]	0.636
InDeg	0.764	0.610 [0.475, 0.727]	0.259 [0.143, 0.367]	0.650
Reach	0.732	0.505 [0.415, 0.593]	0.058 [−0.059, 0.166]	0.302
Topo-QoS	0.553	0.393 [0.275, 0.506]	0.104 [0.016, 0.201]	0.702
Pubs-raw	0.731	0.571 [0.429, 0.699]	—	0.616
Degree-raw	0.199	0.233 [0.102, 0.359]	—	0.719
<i>Learned (seed ensemble)</i>				
GAT-P+InDeg	0.761	0.594	—	0.578
GAT-P-QoS	0.772	0.519	—	0.274
Hybrid-GAT	0.702	0.502	—	0.585
Hybrid-HGT	0.672	0.457	—	0.582
HGT-QoS	0.667	0.440	—	0.201
GAT-QoS	0.647	0.433	—	0.144
HGT-P-QoS	0.618	0.418	—	0.334

$\rho = 0.259$ [0.143, 0.367] with I_{dyn} , and 0.163 [0.043, 0.275] after the first-order expansion is removed instead; **Reach** keeps nothing (0.058, CI includes zero). The count therefore carries some queue-flow signal that the reachability oracle lacks, and transitive reach does not. Two caveats bound this: the sample is the lexicographic 30 Applications per fold, and every learned engine is below **InDeg** on I_{dyn} (0.418–0.594).

The multi-criteria oracle favours different rankers. On I_{comp} , which adds fragmentation, throughput and flow disruption (Section 4.3), QoS-weighted centrality ranks above the count ($\rho = 0.702$ [0.649, 0.753] against 0.650; **InDeg** ahead on only 3 of 12 folds, $p = 0.110$). Raw total degree, the weakest ranker on I^* , reaches 0.719 on I_{comp} (**InDeg** ahead on 4 of 12 folds, Holm $p = 0.220$), and **Reach** falls to 0.302. The learned engines without a centrality prior collapse on this oracle (0.144–0.334); the two hybrids, which carry **Topo-QoS** as a prior, keep 0.58. No ranker is best on all three oracles, so which one to use depends on which failure notion matters (Section 8.1).

Identifying the critical set. Rank correlation over all Applications is not the operational question in a CI gate, which is whether the true top of the ranking gets flagged. Figure 6 gives, for each ranker, the share of the true top-20% set on I^* that falls within its top $k\%$, with ties resolved in expectation (Amendment 12). At $k = 20\%$, **InDeg** recovers 0.49 of the critical set (tie-breaking bounds 0.48–0.50), consistent with its Overlap@K of 0.504: about half of what a top-20% gate flags is not in the true top 20%, and about half of the true top 20% is missed. Recovering 80% of the critical set requires flagging the top 45% by **InDeg** (0.83), the top 40% by **GAT-P-QoS** (0.81) or the top 45% by the first-order expansion (0.89); only the first-order expansion reaches 90% within the top 50%, and **Reach** and **Topo-QoS** do not reach 80% within 50%. On the I_{dyn} sample, **InDeg** reaches 0.83 at 40% and **GAT-P-QoS** 0.65.

7.2. RQ2: What Learned Engines Need

Summary. Learned engines gain +0.08 to +0.11 from reading the dependency graph, where messages reach the scored Applications from their dependents (11–12 of 12 folds), but none exceeds the dependency count. With capacity matched, relation typing adds nothing (typing main effect −0.014). The “QoS” factor cannot be read as QoS content: its inputs include a QoS-weighted in-degree, so switching it off also removes a version of the count.

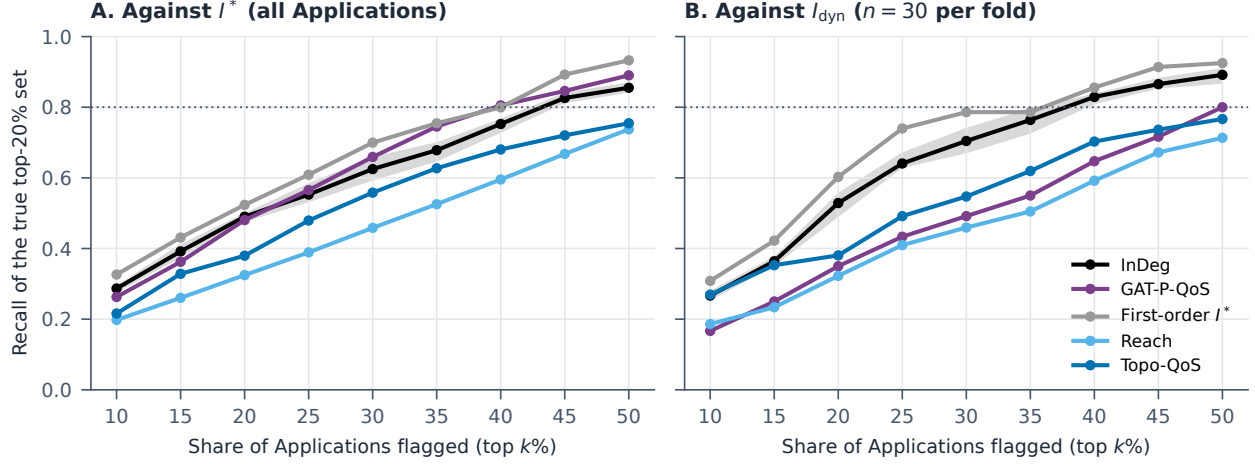


Figure 6: Recall of the true top-20% set (tie-inclusive) by each ranker’s top $k\%$, mean over the twelve LOSO folds, against I^* (left) and the $n = 30$ I_{dyn} sample (right). Ties are resolved in expectation over random tie-breaking; the shaded band gives InDeg’s optimistic and pessimistic bounds. Reach, whose scores tie heavily, has much wider bounds (0.12–0.80 at $k = 20\%$ on I^*), so its expected curve is the only meaningful summary. The dotted line marks 80% recall.

Table 8: The 2×2 with capacity and edge-channel width matched (Amendment 2): GAT ($\neg T \rightarrow Q$, 437,496 parameters), HGT ($T \rightarrow Q$, 434,620), GAT-QoS ($\neg TQ$, 429,992), HGT-QoS (TQ , 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	−0.014	[−0.052, +0.023]	4/12	29.0	0.470	0.940
“QoS” inputs (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ “QoS” interaction	difference of differences	+0.001	[−0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	−0.015	[−0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	−0.013	[−0.054, +0.026]	4/12	27.0	0.380	—
“QoS” inputs, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
“QoS” inputs, typing present	HGT-QoS vs. HGT	+0.073	[−0.002, +0.136]	10/12	19.0	0.129	—

The “QoS” factor is confounded with the count. Its main effect (+0.073) is not significant after Holm correction ($p_{\text{Holm}} = 0.127$), and it could not be attributed to QoS content even if it were. In the node features, w_{in} is the sum of incoming dependency weights, a QoS-weighted version of InDeg; the QoS-off arms zero it, and so remove the single most predictive quantity in the study along with the QoS contracts. A rerun of the 2×2 with w_{in} kept in both arms was not performed. The closed-form controls point the same way: unweighted betweenness (0.591) and constant topic weights (0.595) match or exceed QoS-weighted betweenness (0.553), and permuting QoS profiles across topics changes nothing ($\Delta = -0.006$ [−0.025, +0.015], $p = 0.733$). On I^* we find no evidence that QoS contract content carries ranking signal.

Relation typing under matched capacity. HGT-QoS remains within 0.013 of GAT-QoS (Table 8). On the raw multigraph this says little about typing, because no message reaches an Application there: a gradient-boosted model over the same per-node features without any graph (GBM-Feat, Amendment 8) reaches 0.642, level with GAT-QoS, and removing HGT’s reverse pass, the only route by which messages reach Applications, changes nothing (HGT-QoS-U, −0.010, $p = 0.91$; Supplementary Section S25). On the dependency graph, where messages do arrive, HGT-P-QoS reaches 0.514 with a mean within-fold seed SD of 0.254, against 0.748 and 0.030 for GAT-P-QoS. This is one untuned configuration (width 100, no hyperparameter search), and averaging its five seeds’ predictions raises it to 0.618; we therefore read it as seed instability of that configuration, not as evidence against relational typing in general.

7.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. On five hand-authored models of open-source systems, transitive dependents rank I^* at $\rho = 0.938$ ($\rho_{>0} = 0.871$) and direct dependents at 0.863, without training; the best learned engines reach about 0.80. The models differ structurally from the synthetic corpus in ways that favour **Reach**, and all five were written by one author, so these numbers describe transfer to those models, not to deployed systems.

Table 9: Zero-shot transfer to hand-authored models inspired by five open-source systems (Application population). Models are evaluated out-of-distribution without fine-tuning. $\rho_{>0}$ denotes rank correlation restricted to active components ($I^* > 0$). PR-AUC measures critical-set identification quality. All rows are scored on the same labels (the learned engines’ zero-shot label files; Amendment 12, R5). The training-free rows are not trained on anything, so “zero-shot” applies to the learned rows only. 95% CIs bootstrap the five systems and are indicative only.

Predictor	Evaluation Substrate	Mean ρ [95% CI]	Active $\rho_{>0}$	PR-AUC
Topo	Application layer	0.511 [0.346, 0.703]	−0.104	0.474
Topo-QoS	Dependency graph	0.526 [0.357, 0.699]	−0.088	0.474
Reach	Dependency graph	0.938 [0.879, 0.991]	0.871	0.933
InDeg	Dependency graph	0.863 [0.734, 0.952]	0.321	0.752
HGT-QoS	Raw multigraph	0.760 [0.714, 0.819]	0.236	0.713
GAT-QoS	Raw multigraph	0.805 [0.759, 0.868]	0.319	0.790
Hybrid-HGT	Raw multigraph	0.695 [0.643, 0.730]	0.210	0.602
Hybrid-GAT	Raw multigraph	0.662 [0.597, 0.727]	0.185	0.600
GAT-P-QoS	Dependency graph	0.806 [0.785, 0.829]	0.342	0.838

Reach reaches $\rho = 0.938$ and $\rho_{>0} = 0.871$ on the five models: 0.836–0.997 on the three originally publish–subscribe systems and 0.966–0.998 on the two RPC systems re-expressed as pub-sub meshes (per system: Supplementary Table S36). These values exceed anything **Reach** reaches on the synthetic folds (0.732, $\rho_{>0} = 0.286$), and the difference is structural. Half of the system models’ Applications have zero simulated impact (0.51 on average, against 0.31 on the folds), and their fan-in is more concentrated (Gini 0.65 against 0.50; Supplementary Section S35). On such graphs, separating the inert half from the rest already produces a high full-population ρ , and **Reach**, which is zero exactly for components with no dependents, does that perfectly. The active-stratum $\rho_{>0}$ is the more informative column here, and on it **Reach** (0.871) is far ahead of **InDeg** (0.321 [−0.044, 0.687]) and of every learned engine (0.185–0.342).

7.4. RQ4: Analysis Cost

Summary. The counting path (projection plus **InDeg**) takes 1–29 ms per corpus architecture and 1.1 s at 10,000 components; one labelling pass of the reachability oracle takes 0.08–4.5 s on the corpus and grows roughly quadratically, to 347 s at 2,000 components and 36 minutes at 5,000. The neural pipeline is dominated by feature extraction, which costs median $5.6\times$ a direct run of the oracle; inference itself takes 56 ms at 2,000 nodes. No energy was measured.

Table 10: Per-stage latency of the inference pipeline across graph sizes (CPU, median of 3 runs; 5 for the forward pass). The analysis stage is stable across repeats (p10–p90 within 1% of the median), while the forward pass is dominated by interpreter and dispatch overhead; the 249-node row carries first-call warm-up.

V	E	Analyze	Graph →	HGT	Analyze :	Forward
		(s)	Tensor (s)	Forward (ms)	Forward	p10–p90 (ms)
249	1,127	1.74	0.010	26.5	66×	13.0–34.4
499	2,402	8.32	0.022	16.4	509×	15.6–16.4
999	6,422	44.54	0.056	21.1	2,108×	19.1–36.5
1,998	19,301	239.34	0.157	56.2	4,259×	43.8–57.8

Table 10 breaks down the neural pipeline. Its analysis stage, which computes the node features including betweenness and a directed articulation score, dominates; across the corpus it takes 2.0–17.7× (median $5.6\times$)

Table 11: The counting path against one labelling pass of the reachability oracle I^* on generated graphs of increasing size (Amendment 12, R6; CPU, median of 5 runs; I^* timed once at 4,995 components and not at 9,990, where one pass would take hours). Projection: deriving the Application–Library `DEPENDS_ON` graph from the manifest.

$ V $	$ E $	Projection + InDeg (ms)	Reach (ms)	I^* pass (s)	I^* : count
249	1,081	8.0	11.7	4.0	500×
499	2,437	11.2	24.9	15.1	1,351×
999	6,372	25.3	113.1	58.5	2,313×
1,998	19,242	109.7	490.8	346.6	3,159×
4,995	94,790	426.8	4,369.9	2,163.5	5,069×
9,990	348,277	1,095.3	23,302.4	not timed	—

the time of running I^* directly, so where the I^* ranking itself is wanted, running the oracle is cheaper than approximating it with a learned engine. Table 11 shows the counting path. On the corpus it takes 1–29 ms per architecture against 0.08–4.5 s for one I^* pass, and on generated graphs the gap widens with size: the count grows roughly with the number of edges, I^* roughly with the square of the number of components. At 5,000 components one I^* pass takes 36 minutes and the count 0.43 s. The count’s practical advantage is therefore cost at scale and in inner loops, not accuracy over the oracle it approximates. **Reach** is costlier than **InDeg** (23 s at 10,000 components) because it computes a transitive closure. These are wall-clock measurements on one CPU; no energy was measured, and we make no sustainability claim beyond them.

8. Discussion, Threats to Validity, and Limitations

8.1. Practical Consequences

Why rank at all when the oracle can be run? The reachability oracle I^* reads the same manifest as every ranker in this paper, needs no operational profile, and on the corpus runs in 0.08–4.5 s (Section 7.4). A practitioner who wants I^* can therefore compute it directly, and nothing in this study shows that I^* , or any of its proxies, predicts observed outages. What the study does show is narrower. First, a known architecture metric, afferent coupling computed as a typed two-hop count, reproduces most of I^* ’s ranking ($\rho = 0.764$) at about two orders of magnitude lower cost on the corpus (1–29 ms against 0.08–4.5 s), and the gap widens with size, to over three orders of magnitude at 2,000 components, because I^* ’s labelling cost grows roughly quadratically (Section 7.4). That matters in inner loops, where the ranking must be recomputed for many candidate manifests, and in review settings where an explainable count is preferred to a simulation trace. Second, graph learning adds nothing to that count on this task, on any of the three oracles. Third, the choice among cheap rankers depends on which failure notion the simulator encodes (Table 7), and none of them identifies the critical set sharply.

Choosing a ranker. Table 12 summarises the evidence by oracle. For reachability-style cascade impact, **InDeg** is the default; it keeps queue-flow signal beyond I^* (0.259 partial correlation), which **Reach** does not. On unfamiliar topologies like the five system models, **Reach** separates inert from active components best ($\rho_{>0} = 0.871$), but that result rests on five models by one author. Where failure is understood as fragmentation and throughput loss (I_{comp}), QoS-weighted centrality or even raw total degree ranks better than the counts (0.702 and 0.719 against 0.650). Learned engines are not recommended for any oracle studied here: without a centrality prior they collapse on I_{comp} (0.144–0.334), and none beats **InDeg** on I^* or I_{dyn} .

How much to flag. A top-20% gate on **InDeg** recovers about half of the true top-20% set on I^* (0.49), and the rest of what it flags is not in that set. The recall curves of Figure 6 give the trade-off directly: 80% of the critical set is recovered by flagging the top 45% by **InDeg** (0.83), and 90% is not reached within the top 50%. A gate that must catch most critical components therefore has to flag nearly half the system, which limits the count’s value as a filter; its value is as an ordering for review, not as a pass/fail threshold.

Cost. Pre-deployment analysis avoids provisioning staging clusters for live fault injection [26, 28], but this study measured only wall-clock time, not energy, and makes no sustainability claim beyond it. On wall-clock time, the counting path is the cheapest option by two to three orders of magnitude, direct simulation of I^* is next, and the neural pipeline is the most expensive, because its feature extraction costs median 5.6×

Table 12: Which ranker the evidence supports, by the failure notion the oracle encodes (exploratory; Section 7). Learned engines are omitted: none beats **InDeg** on I^* or I_{dyn} , and without a centrality prior they collapse on I_{comp} .

Failure notion (oracle)	Ranker	Evidence
Reachability cascade (I^*)	InDeg	$\rho = 0.764$, 12/12 folds over Topo-QoS ; recall 0.83 at top 45%
Queue-flow delivery loss (I_{dyn} , $n = 30/\text{fold}$)	InDeg	$\rho = 0.610$; partial 0.259 beyond I^*
Fragmentation and throughput (I_{comp})	Topo-QoS or Degree-raw	$\rho = 0.702 / 0.719$ vs. InDeg 0.650
Unfamiliar topology, inert vs. active	Reach	$\rho_{>0} = 0.871$ on five single-author models

(2.0–17.7 $\times$) a direct run of the oracle it approximates (Section 7.4). Where only the I^* ranking is wanted and the architecture is small, running I^* directly is the simplest choice.

8.2. When Graph Learning Helps, and When It Does Not

Practitioners also need to understand the boundary conditions under which learned models add value, whether relation-specific typing is warranted, and where learning fails. Table 13 summarizes the empirical evidence across accuracy terciles and architectural paradigms (extended per-fold details in Supplementary Section S32).

Table 13: Where graph learning helps. LOSO folds are grouped into terciles of closed-form accuracy (**Topo-QoS** ρ); system models are grouped by architectural paradigm. Mean Spearman ρ against $I^*(v)$, Application population; $\rho_{>0}$: active stratum. For system models, “learned” ranges over the pure learned engines and “closed-form” over **Topo** and **Topo-QoS**. The last column gives training-free dependency counts (Amendment 7) and dependency-graph learners (Amendment 9). Exploratory.

Regime	Where	Best choice	Evidence: raw-multigraph engines	Dependency graph
Closed-form ranks poorly	Microservices, ATM, IoT Smart City, Healthcare	Dependency count or dependency-graph learner	Topo-QoS 0.324; HGT-QoS 0.604, GAT-QoS 0.626, both 4/4 folds. The prior trims the gain (hybrids 0.502 / 0.554).	InDeg 0.693, Reach 0.700, GAT-P-QoS 0.716.
Intermediate	ESB, Telecom RAN, Financial Trading, Industrial SCADA	Dependency count or dependency-graph learner	Topo-QoS 0.561; HGT-QoS 0.589, GAT-QoS 0.660; Hybrid-HGT 0.681, Hybrid-GAT 0.697, both 4/4 folds.	InDeg 0.742, GAT-P-QoS 0.760.
Closed-form ranks well	Logistics Fleet, AV System, Enterprise, Real-Time Gaming	InDeg	Topo-QoS 0.775; HGT-QoS 0.672 (1/4 folds), GAT-QoS 0.620 (0/4); hybrids 0.786 / 0.798.	InDeg 0.858, GAT-P-QoS 0.768.
Unlike the corpus, originally pub-sub	Autoware, EdgeX, Home Assistant	Reach	Learned 0.716–0.927 vs. closed-form 0.289–0.534; learned $\rho_{>0}$ 0.183–0.833 where closed-form is negative.	Reach 0.836–0.997; $\rho_{>0}$ 0.674–0.971.
Unlike the corpus, originally RPC	Online Boutique, Train-Ticket models	Reach	Learned 0.710–0.810, but Topo 0.891 on Online Boutique; learned $\rho_{>0}$ –0.19 to +0.16.	Reach 0.966–0.998; $\rho_{>0}$ 0.813–0.976.

On the raw multigraph, learning operated primarily over node features. Three controls clarify how the raw-multigraph engines functioned. First, gradient-boosted decision trees over per-node features without graph message passing (**GBM-Feat**) achieve $\rho = 0.642$ under LOSO, level with **GAT-QoS** (0.635). Second, on the raw multigraph, all structural relations point away from Applications, meaning standard forward GAT convolutions deliver no messages to the scored nodes: deleting every edge leaves their outputs unchanged. Third, HGT reaches Applications only through reverse edge passes, and ablating them incurs no measurable loss (**HGT-QoS-U**, –0.010, $p = 0.91$; Supplementary Section S25). Thus, on the raw multigraph, neural models acted as learned non-linear combiners of precomputed centralities and degree metrics rather than relational message-passing engines. On the derived dependency graph, however, directed edges flow directly from dependents to dependencies: 3-layer message passing aggregates the true dependent neighborhood, enabling **GAT-P-QoS** to outperform **GBM-Feat** by +0.106 (Supplementary Section S34). Because every learner also receives an in-degree feature (Section 3.5), this margin shows what message passing adds over per-node

features, not that a learner can discover the dependent count unaided; a learner ablation without the in-degree and w_{in} columns was not run.

Homogeneous versus heterogeneous architectures. On the raw multigraph, where message passing into Applications was inactive, typing added nothing measurable (main effect -0.014 , interaction $+0.001$). On the dependency graph, where message passing operates, the heterogeneous transformer (HGT-P-QoS) was unstable across seeds in the single configuration evaluated (width 100, untuned; mean within-fold seed standard deviation 0.254, mean $\rho = 0.514$), whereas the homogeneous GAT-P-QoS was not (0.030, $\rho = 0.748$). This supports a narrow statement: *in this configuration*, relation-specific parameters bought no accuracy and cost stability. It does not show that typing is harmful in general. No typed arm was tuned, averaging five seeds’ predictions lifts HGT-P-QoS to 0.618 (still below InDeg), and denser topologies or architectures whose messages reach scored nodes may behave differently.

8.3. Threats to Validity

Construct validity and oracle circularity. All ground truth is simulated, and no ranker is validated against observed failures. I^* propagates failure along the same subscriber-to-publisher arcs that InDeg counts, so their agreement is largely construction: its first-order expansion reaches 0.808, any component with no dependents receives zero impact, and inert components are separated by construction, which is why $\rho_{>0}$ is reported beside ρ . The two other oracles bound, but do not remove, this threat. I_{dyn} shares the topology and the first-order effect; the partial correlation shows that InDeg carries some I_{dyn} signal beyond I^* (0.259 [0.143, 0.367]), but it is computed on the first 30 Applications per fold in lexicographic order, a sample that may correlate with generation order, and the registered full-population labelling (Amendment 11) was not completed. I_{comp} encodes a different failure notion and ranks the rankers differently. The evaluation is confined to Applications, so dependency Rules 2–4 and 6, which govern brokers and hosts, are untested.

Internal validity, reproducibility, and drift. Predictors consume G_{analysis} while simulation oracles execute on $G_{\text{structural}}$, verified by regression tests. Model capacity, depth and early stopping were held constant across comparison arms, and no arm was tuned (Section 4.2); every statement about a learned engine is therefore conditional on one fixed configuration. The learned engines are also seed-sensitive. Across the five seeds of a fold, the mean within-fold standard deviation of ρ is 0.099 for HGT-QoS, 0.254 for HGT-P-QoS, 0.024 for GAT-QoS and 0.030 for GAT-P-QoS, and single HGT seeds differ from others in the same fold by more than 1.0 (Supplementary Section S35). Averaging the five seeds’ *predictions* instead of their ρ values raises every learned engine, most of all the unstable ones (HGT-P-QoS from 0.514 to 0.618; Supplementary Section S35); the tables report the registered statistic, the mean of per-seed ρ . Across revision cycles and compute devices, learned cells drifted by up to 0.172 (± 0.041 on HGT-QoS), more than several effects this paper reports (the primary contrast, $+0.069$; the QoS main effect, $+0.073$). For that reason every learned cell in Tables 6, 8 and 9 is reported from one named CPU sweep whose per-seed logs reproduce the published mean exactly, and the training-free rows, which do not drift, carry the paper’s conclusions. The Connectivity Degradation Index samples nodes with hash-dependent tie-breaking, so PYTHONHASHSEED=0 is fixed. All reported values are reconciled mechanically against released artifacts.

External validity and the single-modeller threat. Synthetic topologies come from a single generator family, and the five system models are hand-authored approximations (22–41 Applications) of open-source projects, each written by the first author from public documentation. All five RQ3 results are exposed to this threat, and it is not mitigated: no model was authored independently by a second modeller, and no inter-modeller agreement is reported. The system models also differ structurally from the synthetic folds (Supplementary Section S35): a larger share of their Applications has zero simulated impact and their fan-in is more concentrated, which favours Reach and inflates full-population ρ . Two of the five are RPC systems re-expressed as publish–subscribe meshes. Architectures with tens of thousands of components were not evaluated; the counting path was timed up to 10,000 components (Section 7.4), the learned engines and the oracles were not. Published learned criticality models (FINDER, DrBC) were not reproduced, because they do not support typed multigraphs or pub-sub semantics.

Conclusion validity. The only confirmatory results are the null primary contrast, the matched 2×2 and the two hybrid contrasts (Section 6.3). The dependency-count results the paper leads with come from amendments registered after the primary result was known, and are exploratory. LOSO folds share ten

of eleven training scenarios, so the Wilcoxon p -values are nominal and anti-conservative; for training-free rankers the folds are simply twelve scenarios from one generator, so the effective sample is small. With twelve folds, a bootstrap CI over folds is itself coarse, and over five system models it is indicative only.

8.4. Limitations and Future Work

The study leaves open, and states as limitations: validation against observed outages or post-mortems; a hyperparameter search for the learned engines, which were all run in one fixed configuration; a rerun of the 2×2 with the weighted in-degree column kept in both arms, and a learner without in-degree features; the full-population I_{dyn} labels of Amendment 11; an independently authored second model of each open-source system; energy measurement; synchronous call edges, which the dependency rules do not model; and the proposed explanation layer of Section 5, whose attributions have not been validated in any way. Natural next steps are to validate the rankings against historical incident data, to extract architecture graphs automatically from Kubernetes manifests and Helm charts, and to test whether the counts’ advantage survives on oracles that model deadline penalties and buffer overflow explicitly.

9. Conclusion

Software-as-a-Graph (SaG) derives explicit dependency graphs from publish–subscribe deployment manifests and ranks components by simulated cascade impact before deployment. We used it to benchmark closed-form centrality, dependency counts, graph neural networks and hybrid engines on twelve synthetic architectures and five hand-authored models of open-source systems, against three simulators.

The registered primary contrast, a heterogeneous graph transformer against weighted centrality, was null. In exploratory analyses, the simplest ranker won: counting a component’s direct dependents, which is publish–subscribe afferent coupling and a typed two-hop query on the raw graph, ranked reachability-simulated impact at $\rho = 0.764$ and beat centrality on all twelve synthetic architectures. That agreement is largely by construction, because the reachability simulator’s first propagation wave is the same count. On a queue-flow simulator the count kept a partial correlation of 0.259 beyond the reachability oracle, on a sample of 30 Applications per fold; on a multi-criteria simulator, weighted centrality ranked above it. Graph attention networks reading the dependency graph matched the count but never exceeded it, and no learned engine beat it on any oracle. The count identifies the critical set only moderately: catching 80% of the true top fifth requires flagging the top 45%.

For practice, the evidence supports a modest recommendation. Once publish–subscribe dependencies are stated explicitly, afferent coupling is a cheap ordering for architectural review, computable in milliseconds, and it is the baseline that any learned ranker for this task must beat; on this benchmark none did. Whether that ordering, or any simulator it approximates, predicts real outages remains the open question.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision. **Declaration of Competing Interest.** The authors declare no competing financial interests or personal relationships that could have influenced this work. **Funding.** This research received no external grant. **Data Availability.** The replication package (datasets, harnesses, checkpoints, scripts) is available on Zenodo under DOI 10.5281/zenodo.14922108 [97] with `uv/pip` environments. The public repository documents every experiment reported here — protocol, hyperparameters, reproduction command, artifacts and extended results — at <https://github.com/onuralpyigit/software-as-a-graph/tree/main/docs/research/jss/experiments>. Synthetic datasets regenerate byte-identically. The deposit ships all artifacts backing reported tables as a dated bundle (`SaG_JSS_Results_<stamp>`) with a `MANIFEST.json` recording SHA-256 digests, commit hashes, and corpus provenance. Four supplementary artifacts predate provenance stamping and carry no commit or corpus digest: `atm_scale_sweep_v3.json` (S6), `qos_label_ablation.json` (Section 4.3),

threshold_sensitivity_v3.json (S3) and topic_weight_sensitivity_v3.json (S1); their correspondence to the corpus is asserted by the bundle rather than recorded in the file. In the main text, only the two label-ablation correlations of Section 4.3 ($\rho = 0.965$ and 0.977) rest on one of them; the other three back supplementary sensitivity sweeps only, and no number in the abstract or in Tables 6–9 depends on any of the four. The verification script (`reproduce/reconcile_manuscript.py`) runs standalone against the deposit, mechanically verifying 1,347 reported figures in the manuscript and supplement against the JSON artifacts.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic’s Claude to assist with typesetting, LaTeX formatting, the development of analysis and reporting scripts in the replication package, and drafting revisions of the manuscript text in response to review comments. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data Distribution Service (DDS), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, *Microservices: Yesterday, today, and tomorrow*, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye, Service-level fault injection testing, in: *Proceedings of the ACM Symposium on Cloud Computing (SoCC ’21)*, ACM, 2021, pp. 388–402. doi:10.1145/3472883.3487005.
- [20] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: *Proceedings of The Web Conference 2021 (WWW ’21)*, ACM, 2021, pp. 3087–3098. doi:10.1145/3442381.3449905.
- [21] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [22] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [23] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [24] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: *Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006)*, Shaker Verlag, Aachen, Germany, 2006, pp. 161–174.
- [25] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [26] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [27] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/adv.202100707.

- [28] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [29] S. M. Yacoub, H. H. Ammar, A methodology for architecture-level reliability risk analysis, *IEEE Transactions on Software Engineering* 28 (6) (2002) 529–547. doi:10.1109/TSE.2002.1010058.
- [30] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.
- [31] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [32] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [33] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [34] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [35] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [36] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [37] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [38] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [39] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [40] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [41] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [42] X. Meng, P. Shen, Y. Sun, D. Liu, J. Lu, S. Zhang, D. Pei, Microcause: Root cause analysis for microservice systems through graph neural networks, in: *Proc. IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020, pp. 403–414.
- [43] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [44] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [45] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [46] S. Esparrachiar, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [47] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [48] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [49] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [50] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [51] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [52] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [53] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [54] T. Zimmermann, N. Nagappan, Predicting defects using network analysis on dependency graphs, in: *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, ACM, 2008, pp. 531–540. doi:10.1145/1368088.1368161.
- [55] R. Premraj, K. Herzig, Network versus code metrics to predict defects: A replication study, in: 2011 International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, 2011, pp. 215–224. doi:10.1109/ESEM.2011.30.
- [56] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [57] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *arXiv preprint* (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
- [58] A. Santos, A. Cunha, N. Macedo, Static-time extraction and analysis of the ROS computation graph, in: 2019 Third IEEE International Conference on Robotic Computing (IRC), IEEE, 2019. doi:10.1109/IRC.2019.00018.
- [59] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17)*, ACM, 2017, pp. 107–115. doi:10.1145/3143434.3143443.
- [60] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [61] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [62] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and*

- Software 101 (2015) 193–220.
- [63] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
- [64] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
- [65] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [66] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [67] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [68] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
- [69] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
- [70] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
- [71] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.
- [72] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.
- [73] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
- [74] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [75] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
- [76] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
- [77] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
- [78] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
- [79] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
- [80] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [81] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
- [82] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE), 2025, pp. 19–30. doi:10.1145/3676151.3719362.
- [83] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.
- [84] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
- [85] W. Fu, T. Menzies, Easy over hard: A case study on deep learning for software engineering, in: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017), ACM, 2017, pp. 49–60. doi:10.1145/3106237.3106249.
- [86] M. Ferrari Dacrema, P. Cremonesi, D. Jannach, Are we really making much progress? a worrying analysis of recent neural recommendation approaches, in: Proceedings of the 13th ACM Conference on Recommender Systems (RecSys ’19), ACM, 2019, pp. 101–109. doi:10.1145/3298689.3347058.
- [87] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, in: International Conference on Learning Representations (ICLR), 2020. URL <https://openreview.net/forum?id=HygDF6NFPB>
- [88] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
- [89] G. Beliakov, A. Pradera, T. Calvo, Aggregation Functions: A Guide for Practitioners, Vol. 221 of Studies in Fuzziness and Soft Computing, Springer, Berlin, Heidelberg, 2007. doi:10.1007/978-3-540-73721-6.
- [90] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
- [91] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
- [92] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [93] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [94] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [95] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
- [96] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
- [97] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/zenodo.14922108.